

EXPERIMENT 1: Implementing DAC and RBAC in Linux

1. AIM

To understand and apply Discretionary Access Control (DAC) and Role-Based Access Control (RBAC) in Linux by creating users, groups, assigning file permissions, and configuring role-based execution using sudo.

2. THEORY

2.1 Discretionary Access Control (DAC)

DAC means the file owner controls who can read/write/execute a file.
Implemented using:

- Users and Groups
- Permissions (rwx)
- chmod, chown, chgrp

2.2 Role-Based Access Control (RBAC)

RBAC assigns permissions to roles instead of individual users.
Implemented using:

- Groups representing roles
- sudo rules to restrict privileged commands

3. PRE-SETUP STEPS

On Windows Machine:

1. Install VirtualBox or VMware Player
2. Download Ubuntu Desktop ISO
3. Create a VM:
 - RAM: 4GB
 - Disk: 25GB
4. Boot Ubuntu and complete installation

Update Linux

Open Terminal:

```
sudo apt update && sudo apt upgrade -y
```

4. OPERATIONAL STEPS

PART A — Implementing DAC

Step 1: Create Users

```
sudo adduser alice
```

```
sudo adduser bob
```

The `sudo adduser` command is a high-level utility used to create a new user account on the system.

1. **Identity Creation:** It adds a new entry to the `/etc/passwd`, `/etc/shadow`, and `/etc/group` files, establishing **Alice** and **Bob** as recognized system entities.
2. **Home Directory Provisioning:** It automatically creates a personal directory for each user (typically `/home/alice` and `/home/bob`) and populates them with default configuration files (like `.bashrc`).
3. **Group Assignment:** By default, it creates a private group with the same name as the user and adds the user to it.
4. **Security Initialization:** The system will prompt you to set a password for each user, ensuring that their accounts are protected from unauthorized access.

Step 2: Create a Folder

```
mkdir /home/alice/project
```

`mkdir` command stands for "**make directory.**" When you execute `mkdir /home/alice/project`, you are performing a file system operation to create a new folder at a specific location.

Breakdown of the Command

- **mkdir:** The instruction to the operating system to create a new directory.
- **/home/alice/:** The **absolute path** where the folder will be placed. In your setup, this is Alice's personal storage space created during the `adduser` step.
- **project:** The specific name of the new folder being created.

Step 3: Set Ownership

```
sudo chown alice:alice /home/alice/project
```

chown (change owner) command is used to define the administrative and group-level authority over a specific resource.

Breakdown of the Command

- **sudo**: Executes the command with root privileges, as changing ownership of files is a restricted administrative task.
- **chown**: The utility used to change the user and/or group ownership of a file or directory.
- **alice:alice**:
 - The first **alice** specifies the **New Owner** (the individual user).
 - The second **alice** (after the colon) specifies the **New Group** (the primary group associated with the file).
- **/home/alice/project**: The target directory whose ownership is being transferred.

Step 4: Change Permissions

Allow only owner to access:

```
sudo chmod 700 /home/alice/project
```

the chmod command is used to **change mode**, which modifies the access permissions of a file or directory. The value **700** is a specific instruction that tells the operating system exactly who is allowed to interact with that folder.

Breaking Down "700"

The three digits represent three different levels of access: **Owner**, **Group**, and **Others**.

- **7 (The Owner - Alice)**: This is a combination of Read (4) + Write (2) + Execute (1). It gives Alice full control to see files, create new ones, and enter the folder.
- **0 (The Group)**: No permissions. Anyone in the same group as Alice is blocked.
- **0 (Others - e.g., Bob)**: No permissions. Any other user on the system is strictly blocked.

You can verify that this worked by running: `ls -ld /home/alice/project`

The output should start with **drwx-----**.

- The **d** means it's a directory.
- The **rw** shows Alice has full access.
- The remaining **-----** confirms that no one else has any access at all.

Combination	Permissions (rwx)	Professional Use Case
777	<code>rw-rwxrwx</code>	High Risk: Everyone can read, write, and delete. Avoid this in production.
755	<code>rw-r-xr-x</code>	Standard Directory: Owner has full control; others can enter and see files but not change them.
700	<code>rw-----</code>	Private Data: Only the owner can access. Perfect for the "Alice" example in your lab.
644	<code>rw-r--r--</code>	Standard File: Owner can edit; others can only read (e.g., a README file).
600	<code>rw-----</code>	Sensitive File: Only the owner can read/write (used for SSH keys or config files).

In Linux, because each of the three digits (Owner, Group, Others) can be any number from **0 to 7**, there are exactly **512 possible combinations** ($8 \times 8 \times 8$).

Each digit is calculated by adding up the values of the three basic permissions:

- **4** = Read
- **2** = Write
- **1** = Execute

- **0** = No Permission

Step 5: Test

- **Login as Alice** → **Allowed to create files**
- **Login as Bob** → **Should be denied:**

`su - bob`

`cd /home/alice/project`

1. `su - bob`

This command stands for "**Substitute User**" (often referred to as Switch User).

- **The Operation:** It logs you out of your current session and starts a new login session as the user **Bob**.
- **The Hyphen (-):** This is crucial. It tells the system to provide a "login shell," meaning it loads Bob's specific environment variables, paths, and moves you into his home directory (/home/bob).
- **Purpose in Lab:** You are switching identities to test if the restrictions you placed on Alice's folder successfully block other users on the system.

2. `cd /home/alice/project`

This command stands for "**Change Directory.**"

- **The Operation:** Bob is attempting to enter the specific directory that belongs to Alice.
- **The Expected Result:** Because you previously ran `sudo chmod 700 /home/alice/project`, the Linux kernel will check Bob's identity against the permission bits.
- **The Security Outcome:** Since Bob is not the owner (Alice) and has no permissions (the "0" in 700), the system will return an error: **bash: cd: /home/alice/project: Permission denied.**

PART B — Implementing RBAC Using sudo

Step 6: Create a Role Group

`sudo groupadd syshelper`

- **sudo**: Runs the command with administrative (root) privileges, which is required to modify system files like /etc/group.
- **groupadd**: The system utility used to create a new group definition in the Linux operating system.
- **syshelper**: The name of the new group. In your documentation, this represents the "**System Helper**" Role.

Step 7: Add Bob to Role

sudo usermod -aG syshelper bob

- **sudo**: Requires administrative rights to modify system user settings.
- **usermod**: Short for "**user modify**." It is the utility used to change a user's attributes or group memberships.
- **-a (Append)**: This is critical. It tells the system to *add* the user to a new group without removing them from their existing groups. Without this, Bob might lose access to his other assigned groups.
- **-G (Groups)**: Specifies that the following name is the group you want to modify.
- **syshelper**: The name of the group (the Role) you are adding the user to.
- **bob**: The specific user identity being assigned to the role.

Step 8: Assign RBAC Privilege

Open:

sudo visudo

- **sudo**: This command requires root privileges because it modifies /etc/sudoers, the most sensitive security policy file on a Linux system.
- **visudo**: This is a specialized editor for the sudoers file. While it usually opens the file in a text editor (like nano or vi), it is **not** just a text editor.

Add at bottom:

%syshelper ALL=(ALL) /usr/bin/apt

When you run sudo visudo and add the line: %syshelper ALL=(ALL) /usr/bin/apt

You are performing the "Authorization" part of RBAC:

- **%syshelper**: Targets the group (Role) you created earlier.
- **ALL=(ALL)**: Allows this role to act on any host as any user.

- **/usr/bin/apt:** This is the **Principle of Least Privilege**. You are not giving Bob full root power; you are only giving the "sys-helper" role the power to run the apt command.

Step 9: Test RBAC

Login as Bob:

su - bob

sudo apt update

1. su - bob

As discussed previously, this command switches your current session to the user **Bob**.

- **The Goal:** To move from a standard user environment into the specific user environment that has been assigned the **sys-helper** role.
- **The Context:** You are now acting as a limited administrator.

2. sudo apt update

This is the command that tests the **RBAC** rule you added to the sudoers file.

- **sudo:** Bob is requesting to run a command with "SuperUser" privileges.
- **apt update:** This command refreshes the package database on the Linux system.
- **The Logic:** When Bob types this, the system checks the /etc/sudoers file. It sees that Bob is a member of the %sys-helper group and that the %sys-helper group is explicitly allowed to run /usr/bin/apt.

Command	Action	Expected Result	Reason
sudo apt update	Update software	ALLOWED	Authorized in visudo for %sys-helper
sudo useradd	Create new	DENIED	Not authorized in the RBAC

test	user		policy
------	------	--	--------

→ Should work.

Try:

sudo useradd testuser

- **sudo:** Bob is attempting to use "SuperUser" powers to perform an administrative task.
- **useradd:** The Linux utility used to create a new user account on the system.
- **testuser:** The name of the new account Bob is trying to create.

Why this command is **DENIED** for Bob

Even though Bob successfully ran `sudo apt update`, he will receive a "**Permission Denied**" error (or a prompt for a password he doesn't have) when trying to run `useradd`.

1. **Granular Authorization:** In Step 8, you specifically wrote `%syshelper ALL=(ALL) /usr/bin/apt`. This told the system that the **syshelper** role *only* has permission to run the `apt` binary.
2. **Implicit Deny:** In security, anything not explicitly allowed is forbidden. Since `/usr/sbin/useradd` was not included in your `visudo` line, the system blocks Bob from using it.
3. **Principle of Least Privilege:** This demonstrates that Bob has enough power to do his job (updates) but not enough power to compromise the system (creating new users).

→ Should be denied.

5. EXPECTED OUTPUT

- Alice can access `/home/alice/project`
- Bob cannot access it (DAC success)
- Bob can run:
 - `sudo apt update`
- Bob cannot run admin commands (RBAC success)

6. CONCLUSION

This experiment introduced basic Linux access control, showing how DAC restricts file access and RBAC limits privileged operations through roles.